Google Summer of Code 2025 [Sysprof]

Project: Adding eBPF profiling capabilities to Sysprof

GENERAL DETAILS

Name: Varun R Mallya

Time Zone: India (UTC+5.30)

University: Indian Institute of Technology, Roorkee (IIT Roorkee)

Expected Graduation Date: May 2027 (Currently, a Sophomore)

Primary Email: varunrmallya@gmail.com

Secondary Email: varun_rm@me.iitr.ac.in

Matrix ID: @varunrmallya:matrix.org

IRC Nick: varunrmallya

GitHub: varun-r-mallya

GitLab: varunrmallya

Blog: xeon.me

Size of the Project: Long (350 hours)

DESCRIPTION

https://gitlab.gnome.org/Teams/internship/project-ideas/-/issues/51/

This project focuses on enhancing Sysprof by integrating eBPF-based profiling to significantly reduce overhead and improve performance. Currently, Sysprof relies on repeatedly opening and reading numerous `/proc` files to gather process and system data, resulting in frequent syscalls, file descriptor operations, and context switches. These inefficiencies create performance bottlenecks, especially during intensive profiling tasks.

By leveraging eBPF, we can streamline data collection directly within the kernel, eliminating the need for constant `/proc` access and reducing context switches. This approach not only lowers overhead but also ensures atomic and consistent data capture, as eBPF programs can access memory and file-system states without interruption, even in highly dynamic environments.

Additionally, eBPF enables advanced features like monitoring system-wide syscalls and tracking process creation events, providing deeper insights into system behavior. These capabilities were previously difficult to implement with the existing approach. By adopting eBPF, Sysprof will become more efficient, scalable, and feature-rich. This upgrade lays the foundation for future enhancements, making Sysprof a modern and versatile profiling solution.

Mentor

Christian Hergert (`hergertme` on IRC/Matrix or `@chergert` at `gnome.org`)

What city and country will you reside in during the summer?

For most of the summer, I will reside in **Roorkee, Uttarakhand, India** and for some while in **Bangalore, India**. But regardless of where I reside, I will always have access to fast internet.

What applications/libraries of GNOME will the proposed work modify or create?

The proposal aims to transition part of Sysprof's profiling data collection from /proc files to equivalent eBPF programs. This change will primarily involve modifications to libsysprof and libsysprof-capture, along with updates to the corresponding Meson build files. To maintain flexibility, I propose creating a new sub-component called `sysprof-bpf`, allowing both the existing `/proc`-based and the new eBPF-based profiling methods to coexist.

Implementing this will require adding new directories, build rules, and dependencies such as `bpftool` to generate eBPF program skeletons. Additionally, I plan to write comprehensive tests, following the same structure and approach as existing tests in the tests folder, to ensure the reliability and correctness of the new eBPF-based functionality. This approach ensures a smooth integration while preserving backward compatibility and enabling future enhancements.

What benefits does your proposed work have for GNOME and its community?

My proposed work to integrate eBPF into Sysprof offers key benefits for GNOME and its community:

1. Improved Performance and Efficiency: By reducing `/proc` file access and minimizing context switches, Sysprof will become faster and less intrusive, enhancing the profiling experience for developers and users.
2. Enhanced Profiling Capabilities: eBPF enables new features like system-wide syscall monitoring and process creation tracking, providing deeper insights to optimize GNOME applications and the desktop environment.
3. Modernization of Sysprof: Adopting eBPF aligns Sysprof with modern profiling techniques, making it more competitive and versatile, and attracting more contributors to the GNOME ecosystem.
4. Consistent and Accurate Data: eBPF ensures atomic data capture, improving the reliability of Sysprof for debugging and performance analysis.

Why are you the right person to work on this project?

I believe I am a strong fit for this project because of my genuine interest in profiling, my willingness to learn, and my persistent efforts to contribute meaningfully. I was trying to learn methods to improve program performance, which led me to Sysprof. I've also had seniors from my university who have successfully completed a GSoC at GNOME who nudged me towards this program (Divyansh Jain on `Tracker` in 2024, Gurmannat Sohal on `Polari` in 2023 and Patel Jainil on `Pitivi` in 2023). Seeing an open GSoC issue related to eBPF, I decided to contribute, as I find eBPF fascinating and wanted to dive deeper into its applications. Over the past month, I've dedicated significant time to studying eBPF, contributing to Sysprof by identifying and resolving issues, and even preparing to give an introductory lecture on eBPF for my peers at university. I've also built a container runtime from scratch, which helped me understand namespaces and kernel internals.

A key highlight of my contributions was ensuring that a small part of my work was completed ahead of the GNOME 48 Beta release to avoid any blockers. On the technical side, I've used a variety of eBPF maps and have written eBPF programs focused on profiling, such as a cache hit-miss profiler and an strace implementation using eBPF in Rust. I'm confident in handling the eBPF compilation pipeline and am currently learning how to write Meson build configurations for it.

While I'm still building my expertise, I've already gained practical experience with the core aspects of eBPF, including writing programs, compiling them, and exfiltrating data from the kernel to user-space. My proactive approach to learning and problem-solving, combined with my hands-on experience, positions me well to tackle the complexities of integrating eBPF into Sysprof. I'm eager to contribute to this greenfield aspect of the project and am committed to delivering meaningful results.

How do you plan to achieve completion of your project?

Writing the eBPF programs

I plan to create multiple types of counter eBPF programs that trace different types of tracepoints (Also, this multiple program approach aligns with the subsystem

incompatibility of eBPF). I will be defining a base approach to create an eBPF program for counters and then moving on to each kind of counter data required.

Tools that will be used to develop these programs:

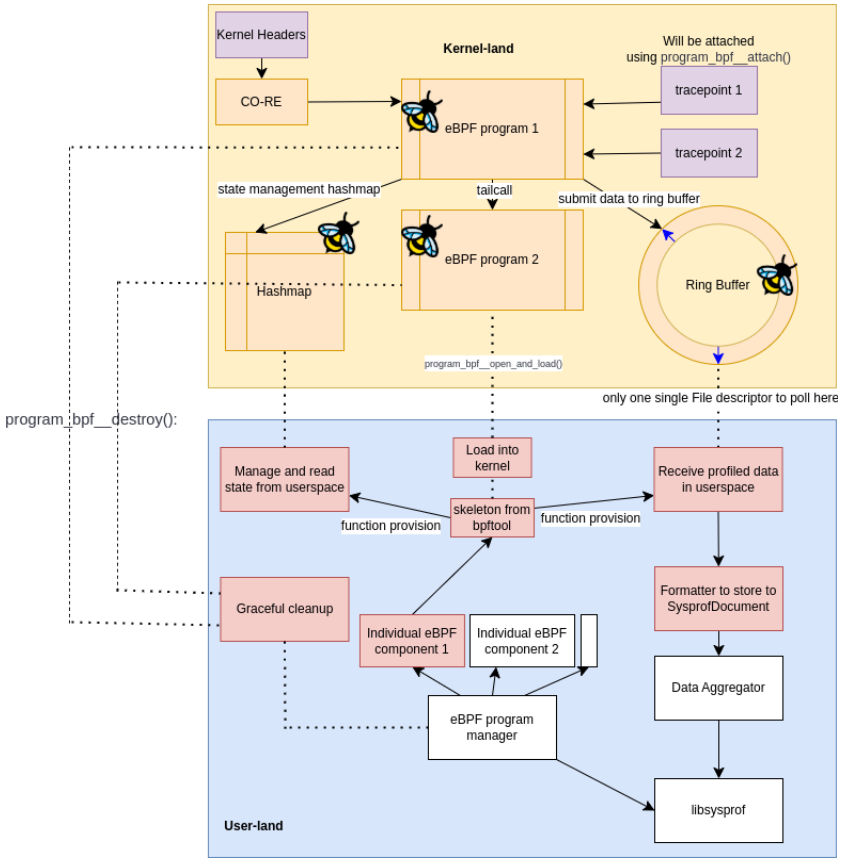
- `libbpf` to ensure simple loading and verification during runtime without additional compilation.
- `bpftool` to generate skeletons and the required `vmlinux.h` file as well. Will also be used to test if programs and maps are loaded into the kernel and to list maps and progs for debugging.
- `bpftop` to monitor CPU usage of these eBPF programs so that they do not affect the profiling process itself.
- `libbpf-tools` as inspiration to find the required tracepoints to trace as well as to test the final product during runtime.
- `clang` with a `-g` flag and -`target bpf` flag as well
- `bpftrace` to test if initial approaches work by writing simple single line eBPF programs for feasibility.
- libbpf-bootstrap & eBPF-C-Template (I authored this by simplifying libbpf-bootstrap) for prototyping and a structure for the final programs.

I will be using the following **features** in the eBPF programs that will collect statistics:

- CO-RE (Compile Once Run Everywhere) using BTF (BPF Type Format to make sure that the types work across kernel versions)
- Skeletons generated by `bpftool` to make loading the program into the kernel and managing it easier.
- Ring Buffers will be used to send data from the kernel in a deferred way to the user-space program through a single file descriptor (per data format) that can be polled instead of the current method in which we possibly open and read hundreds of file descriptors proportional to the number of processes.
- Hashmaps to manage state in the collection programs (to turn part of the collection process on and off based on options set in the GUI)

Base Structure of a single eBPF component along with how it will attach to other parts of Sysprof:

- `program_bpf__open_and_load()`: Opens the eBPF object file and loads it into the kernel.
- `program_bpf__attach()`: Attaches the eBPF program to the appropriate hooks (e.g., tracepoints, kprobes).
- `program_bpf__destroy()`: Cleans up resources when the program exits.



- Blocks in white are managing parts of the sub-program which include an **eBPF program manager** which will be responsible for spawning the required components according to options set in the GUI or CLI. There will also be a **data aggregator** which will be responsible for the similar counting logic that is already implemented in individual files in libsysprof for each component.

- Coming to the actual structure of each eBPF program here, we can have more than 1 program that can transfer to the next using eBPF tailcalls and these will be attached to tracepoints from which data will be collected into defined structs that will be added to a header file to share between the userspace code and the eBPF program as well. When it receives some data, it will check the hashmap for any configuration changes and then move on to correctly formatting and then submitting the data to the ring buffer from which the userspace program will read and add the counter data to SysprofDocument as done in SysprofLinuxInstrument.c.
- We will also be extensively using BPF Timers to poll for data by triggering functions inside the kernel that are not automatically triggered at all times.
- There will also be graceful shutdown and eBPF program unloading that will be done when the profiling is complete. This will be handled individually using simplified functions created in the skeleton of the eBPF program.
- Compile Once Run Everywhere will be used for kernel headers and types to ensure that Sysprof will work across kernel versions.

Checking the requirements of and implementing each eBPF program to replace `/proc` based counters:

My approach will be to analyse each `/proc` file that is being read and finding its function and then trying to find equivalent tracepoints and structs that can be used in its eBPF program. Then, I will be implementing these separately and finally adding them to `sysprof-bpf` along with their corresponding tests.

The following `/proc` files are of importance for profiling in Sysprof:

Found in sysprof-memory-usage.c

`/proc/meminfo`: Extracted values to process include `MemTotal`, `MemFree`, `MemAvailable`, `Buffers`, `Cached`, `SwapCached`, `Mapped`, `Shmem`, `SwapTotal`, `SwapFree`. Derived statistic: `Used memory (total - avail)`. (These calculations are available in the linux kernel source code)

```c
struct sysinfo {
        __kernel_long_t uptime;         /* Seconds since boot */
        __kernel_ulong_t loads[3];      /* 1, 5, and 15 minute load averages */
        __kernel_ulong_t totalram;      /* Total usable main memory size */
        __kernel_ulong_t freeram;       /* Available memory size */
        __kernel_ulong_t sharedram;     /* Amount of shared memory */
        __kernel_ulong_t bufferram;     /* Memory used by buffers */
        __kernel_ulong_t totalswap;     /* Total swap space size */
        __kernel_ulong_t freeswap;      /* swap space still available */
        __u16 procs;                    /* Number of current processes */
        __u16 pad;                      /* Explicit padding for m68k */
        __kernel_ulong_t totalhigh;     /* Total high memory size */
        __kernel_ulong_t freehigh;      /* Available high memory size */
        __u32 mem_unit;                 /* Memory unit size in bytes */
        char _f[20-2*sizeof(__kernel_ulong_t)-sizeof(__u32)];   /* Padding: libc5 uses this.. */
};
```

```c
void si_swapinfo(struct sysinfo *val)
{
        unsigned int type;
        unsigned long nr_to_be_unused = 0;

        spin_lock(&swap_lock);
        for (type = 0; type < nr_swapfiles; type++) {
                struct swap_info_struct *si = swap_info[type];

                if ((si->flags & SWP_USED) && !(si->flags & SWP_WRITEOK))
                        nr_to_be_unused += swap_usage_in_pages(si);
        }
        val->freeswap = atomic_long_read(&nr_swap_pages) + nr_to_be_unused;
        val->totalswap = total_swap_pages + nr_to_be_unused;
        spin_unlock(&swap_lock);
}
```

```c
void si_meminfo(struct sysinfo *val)
{
        val->totalram = totalram_pages();
        val->sharedram = global_node_page_state(NR_SHMEM);
        val->freeram = global_zone_page_state(NR_FREE_PAGES);
        val->bufferram = nr_blockdev_pages();
        val->totalhigh = totalhigh_pages();
        val->freehigh = nr_free_highpages();
        val->mem_unit = PAGE_SIZE;
}
```

On the side are structs and functions relevant to getting this data from the kernel (obtained from the Linux Kernel Source Tree).

We first create a callback function in our eBPF program that runs the `si_meminfo` and `si_swapinfo` functions and dumps data into the `sysinfo` struct and passes the data obtained in this struct to a bpf ring buffer or perf ring buffer to send to user space.

This callback will be called multiple times as scheduled by a bpf timer that will also be a part of the program. We could set the duration between calls to adjust resolution using the config hashmap described in the **base structure** of the eBPF component earlier.

Apart from this, we will also be using BTF enabled kfuncs `kfunc:vmlinux:si_meminfo` and `kfunc:vmlinux:si_swapinfo` to attach to these functions and trace their execution as well.

Found in [sysprof-cpu-usage.c]

- `/proc/stat`: The first line (cpu) represents aggregate CPU usage across all cores. Subsequent lines (cpu0, cpu1, etc.) represent per-core CPU usage.
- **Fields Extracted**:
  - `user`: Time spent in user mode.
  - `nice`: Time spent in user mode with low priority.
  - `sys`: Time spent in system mode.
  - `idle`: Time spent idle.
  - `iowait`: Time spent waiting for I/O.

```c
void kcpustat_cpu_fetch(struct kernel_cpustat *dst, int cpu)
{
        const struct kernel_cpustat *src = &kcpustat_cpu(cpu);
        struct rq *rq;
        int err;

        if (!vtime_accounting_enabled_cpu(cpu)) {
                *dst = *src;
                return;
        }

        rq = cpu_rq(cpu);

        for (;;) {
                struct task_struct *curr;

                rcu_read_lock();
                curr = rcu_dereference(rq->curr);
                if (WARN_ON_ONCE(!curr)) {
                        rcu_read_unlock();
                        *dst = *src;
                        return;
                }

                err = kcpustat_cpu_fetch_vtime(dst, src, curr, cpu);
                rcu_read_unlock();

                if (!err)
                        return;

                cpu_relax();
        }
}
```

    - `irq`: Time spent servicing hardware interrupts.
    - `softirq`: Time spent servicing software interrupts.
    - `steal`: Time spent in other operating systems when running in a virtualized environment.
    - `guest`: Time spent running a virtual CPU for guest operating systems.
    - `guest_nice`: Time spent running a low-priority virtual CPU for guest operating systems.
- `sys/devices/system/cpu/cpu[cpu-id]/cpufreq/scaling_max_freq` and `sys/devices/system/cpu/cpu[cpu-id]/cpufreq/scaling_cur_freq`: These files expose the maximum and current CPU frequencies, respectively, for each CPU core (cpu-id). We can gain access to these functions using the following BTF enabled kprobes and probably kretprobes as well: `kfunc:vmlinux:show_scaling_cur_freq`, `kfunc:vmlinux:show_scaling_max_freq`, `kfunc:vmlinux:store_scaling_max_freq`
- To access CPU statistics and frequency, I will be taking inspiration from here and here. Other kernel functions to trace and structs that might be relevant are listed or shown here as well.
- `kfunc:vmlinux:kcpustat_cpu_fetch` is another tracepoint to keep in mind while processing CPU statistics.
- This component will also be created in a similar manner to the sysprof-memory-usage.c replacement by utilizing BPF Timers to repeatedly poll the kernel for these statistics according to intervals defined in the config map.

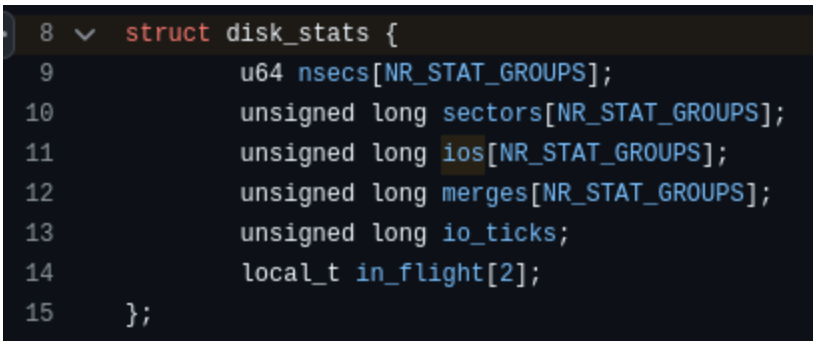
```
36    SEC("tp_btf/cpu_frequency")
37    int BPF_PROG(cpu_frequency, unsigned int state, unsigned int cpu_id)
38    {
39            if (filter_cg && !bpf_current_task_under_cgroup(&cgroup_map, 0))
40                    return 0;
41
42            if (cpu_id >= MAX_CPU_NR)
43                    return 0;
44
45            clamp_umax(cpu_id, MAX_CPU_NR - 1);
46            freqs_mhz[cpu_id] = state / 1000;
47            return 0;
48    }
49
```

```
19  enum cpu_usage_stat {
20          CPUTIME_USER,
21          CPUTIME_NICE,
22          CPUTIME_SYSTEM,
23          CPUTIME_SOFTIRQ,
24          CPUTIME_IRQ,
25          CPUTIME_IDLE,
26          CPUTIME_IOWAIT,
27          CPUTIME_STEAL,
28          CPUTIME_GUEST,
29          CPUTIME_GUEST_NICE,
30  #ifdef CONFIG_SCHED_CORE
31          CPUTIME_FORCEIDLE,
32  #endif
33          NR_STATS,
34  };
35
36  struct kernel_cpustat {
37          u64 cpustat[NR_STATS];
38  };
39
40  struct kernel_stat {
41          unsigned long irqs_sum;
42          unsigned int softirqs[NR_SOFTIRQS];
43  };
```

Found in [sysprof-disk-usage.c]

- `/proc/diskstats`: Major and minor device numbers. Device name (e.g., sda, nvme0n1).
- **Fields Extracted:**
  - `reads_total`: Total number of reads completed.
  - `reads_merged`: Number of reads merged.
  - `reads_sectors`: Number of sectors read.
  - `reads_msec`: Time spent reading (in milliseconds).
  - `writes_total`: Total number of writes completed.
  - `writes_merged`: Number of writes merged.
  - `writes_sectors`: Number of sectors written.
  - `writes_msec`: Time spent writing (in milliseconds).
  - `iops_active`: Number of I/O operations currently in progress.
  - `iops_msec`: Time spent doing I/O operations.
  - `iops_msec_weighted`: Weighted time spent doing I/O operations.
- `kfunc:vmlinux:diskstats_show` would be the required kfunc here and `disk_stats` is the struct that will be filled from this.
- I will also be taking inspiration from libbpf-tools like biolatency and biopattern.

- `block:block_rq_issue`: This tracepoint is triggered when a block I/O request is issued to a storage device. It provides information about the request, including the device, sector, number of sectors, and whether it's a read or write operation.
- `block:block_rq_complete`: This tracepoint is triggered when a block I/O request is completed. It provides information about the outcome of the request, including whether it was a read or write, the number of sectors transferred, and the time taken.



```
8  struct disk_stats {
9          u64 nsecs[NR_STAT_GROUPS];
10         unsigned long sectors[NR_STAT_GROUPS];
11         unsigned long ios[NR_STAT_GROUPS];
12         unsigned long merges[NR_STAT_GROUPS];
13         unsigned long io_ticks;
14         local_t in_flight[2];
15 };
```

Found in [sysprof-network-usage.c]

- `/proc/net/dev`: Contains statistics for a specific network interface
- **Fields Extracted:**
  - `rx_bytes`: Total bytes received.
  - `rx_packets`: Total packets received.
  - `rx_errors`: Receive errors.
  - `rx_dropped`: Dropped packets during reception.
  - `tx_bytes`: Total bytes transmitted.
  - `tx_packets`: Total packets transmitted.
  - `tx_errors`: Transmit errors.
  - `tx_dropped`: Dropped packets during transmission.
  - Other fields include FIFO errors, frame errors, collisions, and compressed packets.

The required kernel tracepoints are listed along with their functions here.

- `net:netif_receive_skb`: Triggered when a packet is received by the network interface.
- `net:net_dev_start_xmit`: Triggered when a packet is transmitted.
- `net:net_dev_xmit`: Triggered when a packet transmission is completed.

From these tracepoints, some basic calculations can yield the required statistics. These are available as both kfuncs and tracepoints, so they can be both called as well as observed directly.

Found in [sysprof-linux-instrument.c]

Opens for every process that is currently running

- `/proc/[pid]/maps`: Memory mappings of a process (used in `add_mmaps`).
- `/proc/[pid]/cmdline`: Command line of a process (used in `add_process_info`).
- `/proc/[pid]/comm`: Name of the process (used in `add_process_info`).
- `/proc/[pid]/mountinfo`: Mount points of a process (used in `add_process_info` and `process_started_cb`).
- `/proc/[pid]/cgroup`: cgroup information (used in `populate_overlays`).
- `/proc/[pid]/root/.flatpak-info`: Flatpak-specific information (used in `populate_overlays` and `process_started_cb`).
- `/proc/[pid]/root/run/.containerenv`: Podman-specific information (used in `populate_overlays`).
- `/proc/cpuinfo`: CPU information (used in `sysprof_linux_instrument_prepare_fiber`).
- `/proc/mounts`: Mounted filesystems (used in `sysprof_linux_instrument_prepare_fiber`).

Quoting the kernel documentation here:
“ For example, users can define a BPF iterator that iterates over every task on the system and dumps the total amount of CPU runtime currently used by each of them. Another BPF task

iterator may instead dump the cgroup information for each task. Such flexibility is the core value of BPF iterators. “

I haven’t studied bpf iterators yet, so this is something that I will be building alongside studying it as well. This part is still a work in progress, so I will be planning on how to build this in the initial few weeks of the GSoC period.

Information like `cmdline` and `comm` can be obtained from structs while tracing `tracepoint:syscalls:sys_enter_execve` and the corresponding `tracepoint:syscalls:sys_exit_execve`.

Optional goals

Apart from these, if time permits, we could add tools to do the following as the framework to add more eBPF programs and components will already be set up. Currently, these are a bit harder to achieve using the perf subsystem, but once we add in eBPF capabilities to Sysprof, these will be much easier to add.

Cache hit or miss profiler

This would be useful in profiling programs inside a tool like Sysprof as it can show which functions in the programs to be profiled are the most inefficient in terms of cache usage. It is not yet clear if eBPF can provide this functionality for individual programs, but it can certainly do this for the full system by observing tracepoints that deal with cache and paging.

strace like tool

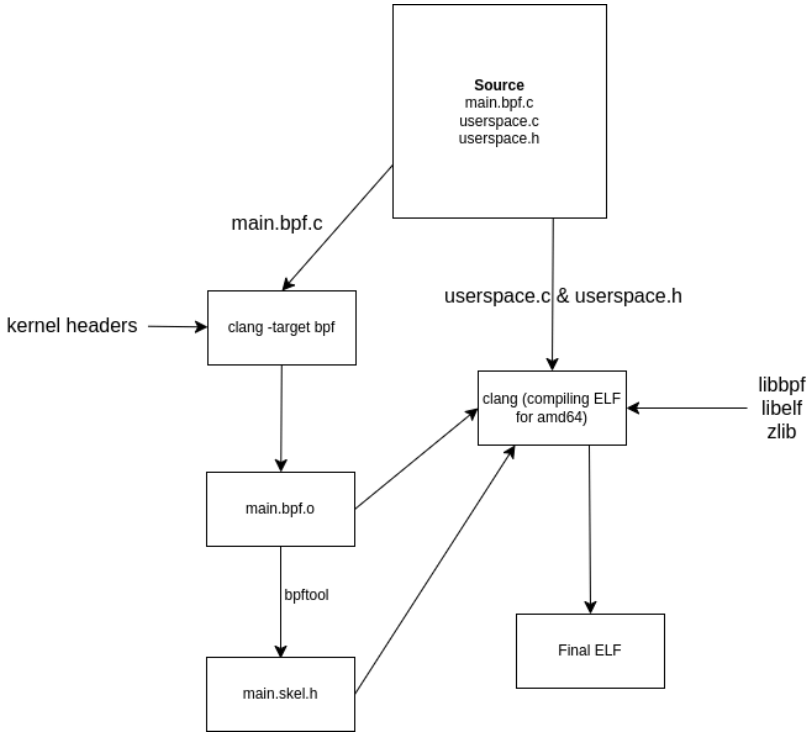
This can be used to monitor syscalls that a program sends out. Would be incredibly easy to implement with eBPF for sampling across the system (I already have a basic implementation ready for this here) Could be used to profile programs which might need to be optimised to reduce the amount of context switches arising due to syscalls (Like Sysprof itself, in its current state)

Writing Meson build configurations to add `sysprof-bpf` to `libsysprof`

The following describes the eBPF program compilation process that I will be implementing into Sysprof:

eBPF program compilation process

A dependency of bpftool will be added to the current compilation process, then `vmlinux.h` is generated using bpftool or a general version is used from https://github.com/libbpf/vmlinux. The eBPF programs are first compiled and their object files stored. Then, the skeleton headers of the required eBPF programs will be created and stored temporarily in the build directory. Then, the actual user-space code will be compiled alongside the rest of the `*.c` files in Sysprof and the required libraries as mentioned (libbpf, libelf and zlib) will be compiled as well. The final binary will not require any eBPF compiler during runtime.



Adding options to the UI and the CLI to profile with eBPF

I have previously worked on adding options for new features on the UI and CLI and also helped out with persistence of those options on the UI as well.

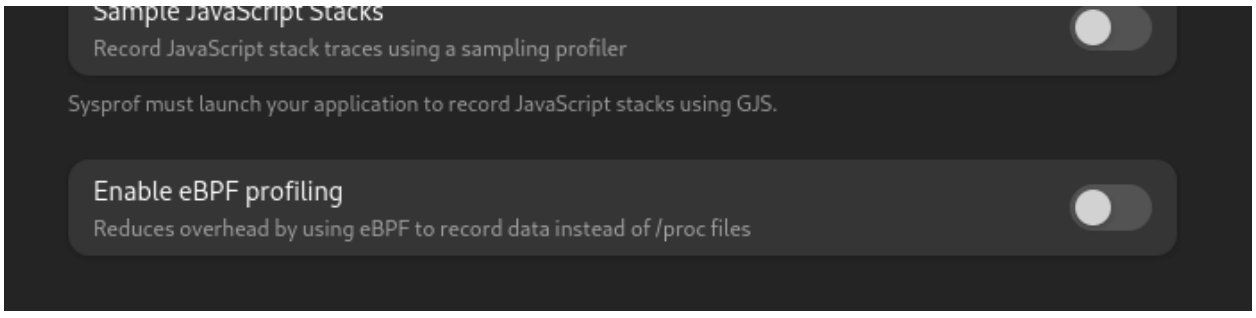
I will be first creating a `SysprofInstrument` inside `sysprof-bpf` and then enabling that instrument in sysprof-recording-template.c and every capability provided by sysprof-bpf will be activated while corresponding capabilities currently provided using `/proc` will be deactivated.

```
620   if (self->power_profile && self->power_profile[0])
621     sysprof_profiler_add_instrument (profiler, sysprof_power_profile_new (self->power_profile));
622
623   if (self->ebpf_profiling_enabled)
624     sysprof_profiler_add_instrument (profiler, sysprof_ebpf_profiling_new ());
625
626   if (self->battery_charge)
627     sysprof_profiler_add_instrument (profiler, sysprof_battery_charge_new ());
628
629   if (self->bundle_symbols)
```

Then, I will be adding UI options with a GTKSwitch and corresponding changes to activate and deactivate this instrument.

```
48    GtkListBox              *sidebar_list_box;
49    AdwPreferencesPage      *record_page;
50    GtkListBox              *app_environment;
51    GtkSwitch               *sample_native_stacks;
52    GtkSwitch               *enable_ebpf_profiling;
53    GtkSwitch               *sample_javascript_stacks;
54    GtkSwitch               *record_disk_usage;
55    GtkSwitch               *record_network_usage;
56    GtkSwitch               *record_compositor;
```

```
175   <object class="AdwPreferencesGroup">
176     <child>
177       <object class="AdwActionRow">
178         <property name="activatable-widget">enable_ebpf_profiling</property>
179         <property name="title" translatable="yes">Enable eBPF profiling</property>
180         <property name="subtitle" translatable="yes">Reduces overhead by using eBPF to record data instead of /proc files</property>
181         <child type="suffix">
182           <object class="GtkSwitch" id="enable_ebpf_profiling">
183             <property name="valign">center</property>
184             <property name="active" bind-source="recording_template" bind-flags="bidirectional|sync-create" bind-property="ebpf_profiling
185           </object>
186         </child>
187       </object>
188   </child>
```



The end result of the feature on the UI will be as shown above. There will also be a corresponding option added in `sysprof-cli` and conditionally in `sysprofd` if time permits.

The header files of sysprof-bpf will also be included in sysprof.h to make it available across the whole program and corresponding issues will be solved which arise from its addition.

In addition to this, I would need to study the documentation of `libdex` to be able to implement asynchronously executing instruments using eBPF: for example, the loops that I will be using to poll the file descriptors of the eBPF maps of each of the components.

Please provide a sequence of tasks and subtasks and how long (days) you estimate it will take you to complete each of them. Highlight important milestones/deliverables.

Pre GSOC Period	
April 1 - May 4	• I would try completing the remaining issues that I am working on in Sysprof before the community bonding period and also study a little more about eBPF iterators and timers which will be used extensively here.
Community Bonding Period	
May 15 - May 28	• With the help of my mentor, I'll familiarize myself with the codebase and think of improving on my current structure of code. I will also be learning the format in which the data from each of the components will be added to SysprofDocument. • I'll also be testing out the individual eBPF programs for validity as well as feasibility here. • Verify the tracepoints and finalise the structure of sysprof-bpf so that I can program with a goal in mind.
Coding Period	
June 2 - June 9	• Writing the eBPF component for memory information and individually testing it. • Writing Meson Build configs for Sysprof and creating a SysprofInstrument after making Sysprof-BPF, ie. a method to convert the data received from the eBPF program into the standard format. Setting up the full compilation process and adding dependencies.

	● Fully ensuring memory information access inside libsysprof.
June 10 - June 20	Work on ● Writing the eBPF components for CPU, Disk/IO and Network statistics and individually testing them. ● Adding them to the framework made previously and testing their access inside libsysprof.
June 21 - July 2	Work on ● Writing GUI and CLI options to turn the eBPF based SysprofInstruments on and off. ● Buffer time to solve any issues arising from the previous part as well as to ensure full completion of the previous part.
June 3 - July 14	Work on ● Writing the eBPF component to monitor mount options, memory mappings and the container side of things as well for all processes. This will be individually tested right now, but not added to sysprof-bpf. ● Prepare for mid-evaluation
Mid evaluation	
July 18 - July 24	● Add the per-process monitoring component made before mid-evaluation to Sysprof and test for access and correct formatting of data.
July 25 - August 5	Once the above is successfully completed and implementation tested minimally, ● Write eBPF components for syscall tracing and cache hit-miss profiling and individually test. ● Also debug the previously written eBPF programs for issues as well as thoroughly checking that the compilation process works across kernel and OS versions.
August 6 - August 16	● Adding the syscall tracing and cache profiling features to the SysprofDocument format as well as adding a section for them in the UI. ● Adding any more sections in the UI and CLI for anything that might be required to make this feature more usable.
August 17 - September 1	Work on ● Pending or last-minute issues and preparing for final

	evaluation.
Final evaluation	

What are your past experiences with the open source world as a user and as a contributor?

My journey in the open-source world began as a user at 14, when I installed Raspbian on an old laptop because I did not have a Raspberry PI. This eventually led me to explore various GNU/Linux distributions because I knew how to install them now and I eventually started using GNU/Linux instead of Windows full time. Then, I started building small programs to assist with homework, relying on open-source tools like Geogebra and Python. My interest in hardware also led me to use open-source software for Arduino projects, building 3D printers, and open source PCB designing software to design circuit boards for amplifiers and radio transmitters.

In college, I joined SDSLabs, a student-run open-source group, where I transitioned from a user to a contributor. I began contributing to projects I loved, such as exploring gcc-rs (GCC's Rust compiler), engaging with the Seanime community (an open-source anime streaming platform), and experimenting with fuzzing through LibAFL and trying to contribute over there. Recently, as I delve into eBPF, I've been interacting with the `bpftool` community and am on the verge of making my first commit. These experiences have deepened my understanding of open-source development and fueled my passion for contributing to impactful projects.

Please include links to your code contributions which have already been merged, or to Gitlab merge requests for the issues you fixed for the project of your proposal or any other GNOME projects. This demonstrates your willingness to learn and familiarity with development workflow.

Issues I created and solved:

- https://gitlab.gnome.org/GNOME/sysprof/-/issues/136
- https://gitlab.gnome.org/GNOME/sysprof/-/issues/132

Merge Requests that have been merged:

- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/117
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/118

- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/120
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/125
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/122
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/128
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/137
- https://gitlab.gnome.org/GNOME/sysprof/-/merge_requests/136

Issues I am currently working on actively:

- https://gitlab.gnome.org/GNOME/sysprof/-/issues/120

  Trying to add a filtering function which will filter by PID so that individual processes can be analysed keeping only that as the root.

- https://gitlab.gnome.org/GNOME/sysprof/-/issues/83

  Adding a split functionality in the CLI which will split the Syscap file with time range.

- Research on adding GPU Shader profiling capabilities to Sysprof

  This explores adding GPU shader profiling capabilities to Sysprof, a new area for the tool. I am currently searching for methods to achieve this and documenting my progress on my blog at xeon.me/posts. The work is closely tied to this merge request in Mesa, where I am investigating ways to extract data from the graphics driver. This will enable Sysprof to profile loaded shaders, identify bottlenecks, and diagnose performance issues, expanding its utility for GPU-focused performance analysis.

If available, please include links to any code you wrote for other open source projects.

- https://github.com/libbpf/bpftool/commit/1c0999a0327aee51b0995062b0936c6b01cd1725

  This points to a potential contribution on `bpftool` that will be solving the long term pending issue of completions on the `zsh` shell.

- https://github.com/sdslabs/Helios/commit/0cb166bf688d2f5f1567772ccb8dd94f0fa50dfe
  Contributions on Quizio, a quizzing application written in Typescript.
- https://github.com/Rust-GCC/gccrs/pull/3698 and https://github.com/Rust-GCC/gccrs/pull/3697

  Contributions to GCC's new Rust compiler - GCC Rust where I tried to remove the EnumItemDiscriminant class according to the new standard.

- https://github.com/IMRCLab/libmotioncapture/pull/23 Bug fixes to `libmotioncapture`, a motion tracking library.
- https://github.com/varun-r-mallya/envvar A library I created to ensure environment variable parsing in Elixir.

What other relevant projects have you worked on previously and what knowledge have you gained from working on them?

I've worked on several projects that have equipped me with relevant knowledge and skills to deliver on this proposal. One of the most significant was building a container runtime from scratch, which you can find here. This project taught me about syscalls, namespaces, and cgroups, and gave me a solid introduction to Linux kernel internals. Additionally, I completed the first few labs of the xv6 Operating Systems course, available here, which helped me understand operating systems at a deeper level and debug a toy kernel using QEMU.

On the eBPF side, I'm halfway through **Learning eBPF** by Liz Rice and have completed several exercises, which are available here. I've also built an implementation of strace using eBPF, which you can find here. This project involved writing eBPF programs to trace syscalls, giving me hands-on experience with eBPF maps, kernel-to-user-space data exfiltration, and the eBPF toolchain.

These experiences have given me a strong foundation in Linux kernel internals, system-level programming, and eBPF, all of which are directly relevant to this project

In addition to this, I've worked with JavaScript, TypeScript, Elixir, Go, Rust, SQL, MongoDB and C++ for multiple projects like web-servers, websites, games with Godot, blockchain smart contracts, toy interpreters and emulators. I have a slight interest in AI for which I learned to train my own models using Tensorflow and Pytorch as well. I learned a lot of

maths from that phase which turned out to be pretty useful for me. I've also built a developer toolchain for a blockchain data storage company called Walrus and an MVP for an AI startup neander.ai. I have also done a small amount of reverse engineering in Capture The Flag contests, so I have a pretty ok understanding of low level programming concepts.

I've also won hackathons like ETHIndia 2024 (a blockchain and web3 hackathon), ETHOnline 2024 and PiWOT India 2025. I am also Rank 2 for the Indian region in CSAW-Embedded Security Challenge conducted by NYU Tandon where I solved hardware and audio challenges.

What other time commitments, such as school work, exams, research, another job, planned vacation, etc? What are the dates for these commitments and how many hours a week do these commitments take?

I can dedicate more than 40 hours a week to this project. My college's summer vacations span from 20.05.2025 to 16.07.2025, during which I will have no classes or academic commitments, allowing me to devote 45+ hours weekly. I have my end-term examinations from 05.05.2025 to 14.05.2025, during which I will be unavailable, but I will be completely free afterward.

Once classes resume, I can easily dedicate 25 hours a week. Although classes will occupy a major part of my day, GSoC will remain a very important priority. I generally work from 7 PM to 2:30 AM IST but can adjust my schedule if needed. Other than this project, I have no other commitments. I will maintain transparency with the community and keep everyone updated on my availability. I am fully committed to this project, both in time and effort. Also, this is the only project I've applied to.